# Introduction to Neural Networks

Jugal Kalita
University of Colorado Colorado Springs

## Specifying a Function in a Table

We can easily specify a function in a table given in an app like Excel.

In the top two tables, we have four data examples, each with two features or attributes. There is a column at the end that computes a linear function of the features.

The last column of each of the two lower tables computes a simple non-linear function of the features.

| $x_1$ | $x_2$ | $f(x_1, x_2) = x_1 + x_2$ |
|---|---|---|
| 2 | 3 | 5 |
| 5 | 7 | 12 |
| 2 | 0 | 2 |
| 9 | 6 | 15 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = 2x + 3x_2 + 1$ |
|---|---|---|
| 2 | 3 | 14 |
| 5 | 7 | 32 |
| 2 | 0 | 5 |
| 9 | 6 | 37 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = x_1 \times x_2$ |
|---|---|---|
| 2 | 3 | 6 |
| 5 | 7 | 35 |
| 2 | 0 | 0 |
| 9 | 6 | 54 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = x_1^2 + x_2^2$ |
|---|---|---|
| 2 | 3 | 13 |
| 5 | 7 | 74 |
| 2 | 0 | 4 |
| 9 | 6 | 117 |

## Machine Learning: Empirically Discovering a Function in a Dataset

Now suppose, we are not given the function. We need to discover the function.

In each top table, there is a linear functions to discover.

In each bottom table, there is a non-linear function to discover.

Discovering functions from data is **machine learning**. The learned function can be used to "predict" values for unseen arguments (parameters)

Discovering functions like these ones where the resulting value is numeric is called **Regression**.

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 5 |
| 5 | 7 | 12 |
| 2 | 0 | 2 |
| 9 | 6 | 15 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 14 |
| 5 | 7 | 32 |
| 2 | 0 | 5 |
| 9 | 6 | 37 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 6 |
| 5 | 7 | 35 |
| 2 | 0 | 0 |
| 9 | 6 | 54 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 13 |
| 5 | 7 | 74 |
| 2 | 0 | 4 |
| 9 | 6 | 117 |

## Machine Learning

In machine learning, we have a dataset that gives us features (observations, measurements, etc.,) about data examples. We assume that there is an unknown function of the features hidden in the dataset.

We need to discover or learn it.

The function can be linear or non-linear, usually highly non-linear.

The learned function than can be used to obtain values for new previously unseen data examples or samples.

The example functions have two parameters or arguments or variables or features.

In real examples, the number of features can be in the tens, hundreds, thousands, tens of thousands, millions or even billions of parameters.

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 5 |
| 5 | 7 | 12 |
| 2 | 0 | 2 |
| 9 | 6 | 15 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 6 |
| 5 | 7 | 35 |
| 2 | 0 | 0 |
| 9 | 6 | 54 |

# Machine Learning

The features are observations or measurements, and therefore can be observed or measured or transcribed incorrectly. The original table is given first and then the "actual" table that results from measurements and from which we have to learn.

Some measurements can be quite good, others so so, and a small number can be quite off or can be outliers.

We still need to find the function.

The number of possible functions to discover is infinite for us to try out various functions exhaustively. We need to find the "best" match according to some criteria.

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 5 |
| 5 | 7 | 12 |
| 2 | 0 | 2 |
| 9 | 6 | 15 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2 | 3 | 6 |
| 5 | 7 | 35 |
| 2 | 0 | 0 |
| 9 | 6 | 54 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2.05 | 3.11 | 5.02 |
| 5.01 | 6.99 | 11.75 |
| 2.13 | 0.01 | 2.31 |
| 9.00 | 6.00 | 15.01 |

| $x_1$ | $x_2$ | $f(x_1, x_2) = ?$ |
|---|---|---|
| 2.11 | 3.05 | 6.07 |
| 5.03 | 7.07 | 34.00 |
| 1.95 | 0.02 | 0.11 |
| 9.00 | 6.00 | 60.05 |

# Machine Learning: Scaling up

The Boston Housing Dataset

The Boston Housing Dataset is a derived from information collected by the U.S. Census Service concerning housing in the area of Boston MA. The following describes the dataset columns:

- CRIM - per capita crime rate by town
- ZN - proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS - proportion of non-retail business acres per town.
- CHAS - Charles River dummy variable (1 if tract bounds river; 0 otherwise)
- NOX - nitric oxides concentration (parts per 10 million)
- RM - average number of rooms per dwelling
- AGE - proportion of owner-occupied units built prior to 1940
- DIS - weighted distances to five Boston employment centres
- RAD - index of accessibility to radial highways
- TAX - full-value property-tax rate per $10,000
- PTRATIO - pupil-teacher ratio by town
- B - 1000(Bk - 0.63)^2 where Bk is the proportion of blacks by town
- LSTAT - % lower status of the population
- MEDV - Median value of owner-occupied homes in $1000's

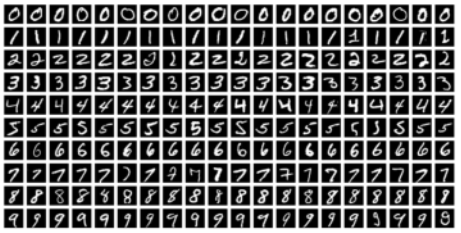
```
['housing.csv']
      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296.0
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242.0
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242.0
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222.0
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222.0

   PTRATIO       B  LSTAT  MEDV
0     15.3  396.90   4.98  24.0
1     17.8  396.90   9.14  21.6
2     17.8  392.83   4.03  34.7
3     18.7  394.63   2.94  33.4
4     18.7  396.90   5.33  36.2
```

This dataset contains information collected by the U.S Census Service concerning housing in the area of Boston Mass. The dataset is small in size with only 506 cases.

The data was originally published by Harrison, D. and Rubinfeld, D.L. `*Hedonic prices and the demand for clean air*', J. Environ. Economics & Management, vol.5, 81-102, 1978.

# Machine Learning: Scaling up

The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems.



The MNIST database contains 60,000 training images and 10,000 testing images.

Each sample is a 28x28 image such that the center of mass of the pixels is in the center of the image.

We can think of each digit as a data sample with 28x28=784 numbers between 0 and 255, with a label (the actual digit) as the last column. Here, the goal would be to find the function in this table that separates the digits from one another as accurately as possible.

# Artificial Neural Networks Learn Approximate Functions

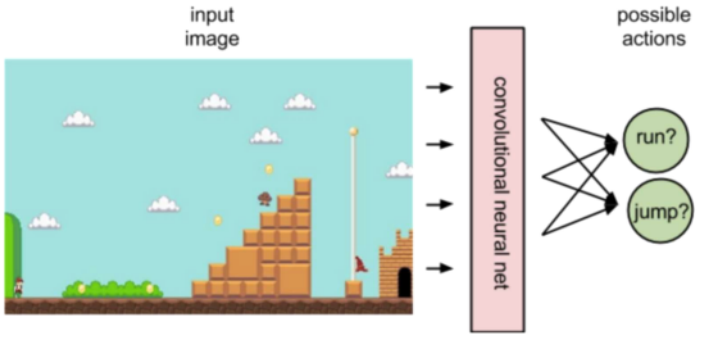
- Example: Classification: **x** is an image, and **y** is label or type of image.

$f^*($  $) =$ **cat**

$f^*($  $) =$ **dog**

# ANNs in Reinforcement Learning

- Example: Classification: **x** is an image, and **y** is the action to perform.



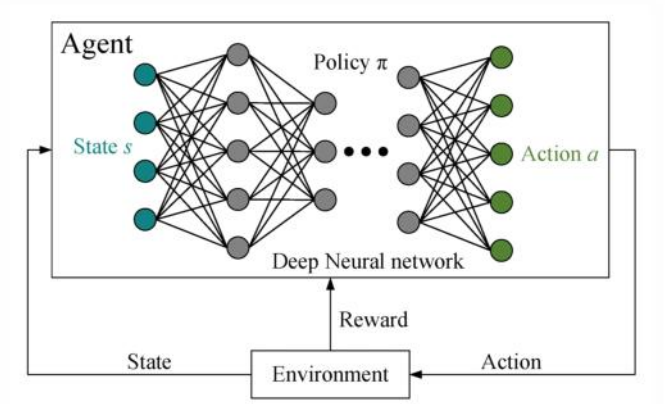

From wiki.pathmind.com

# Biological Inspiration

- Computer programs are brittle, they break often, and they cannot solve many problems humans solve easily and reliably.
- We want to make the computer more robust, intelligent, and endow it the ability to learn.
- An approach: Model computer software (and/or hardware) nominally after the brain.

# ANNs in Reinforcement Learning

Generic policy learning neural network

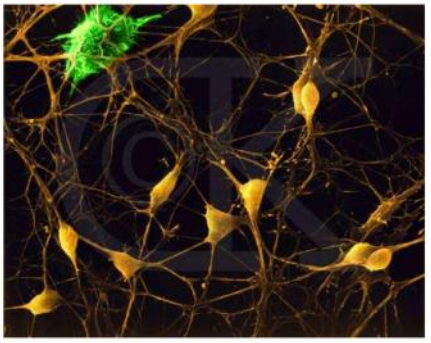



From https://www.mdpi.com/2076-3417/13/16/9467

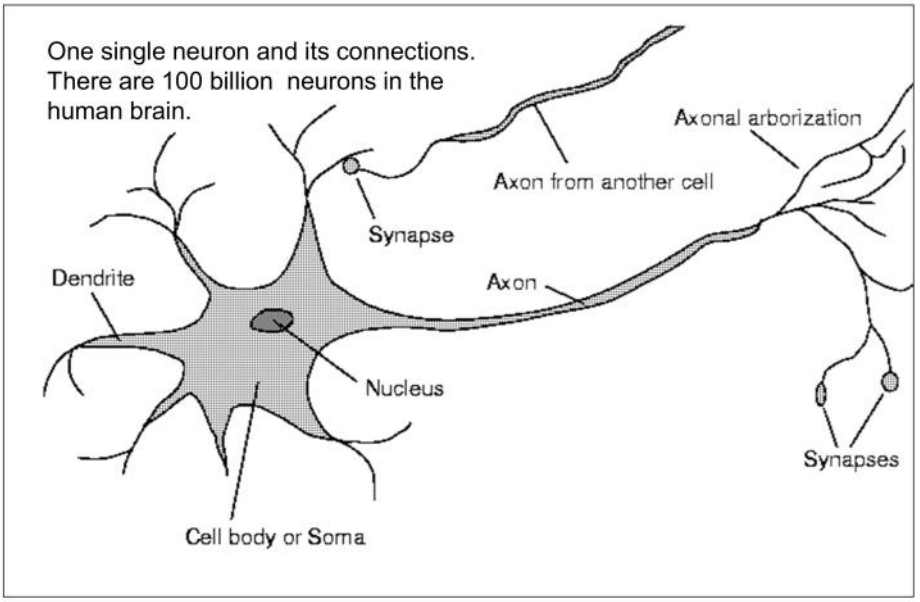
# Introduction to Neural Networks

# Neurons in the Brain

- Although heterogeneous and composed of many complex components, at a low level the brain is composed of neurons, ~100 billion of them, each connected to 3-4000 others on average.



# The Brain is a Network of Neurons

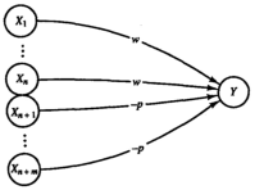


# Natural Neural Networks

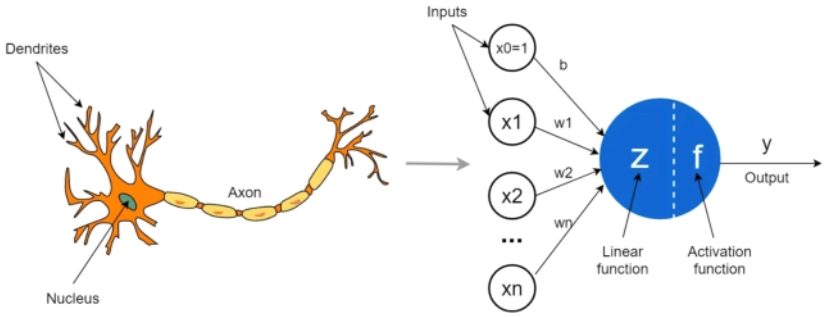
- Signals “move” via electrochemical pulses, combination of electrical and chemical signals working together.
- A neuron receives input from other neurons (maybe hundreds or even thousands) from its synapses, 3-4000 on average.
- The synapses release a chemical transmitter – the sum of which can cause a threshold to be reached – causing the neuron to “fire”.
- In other words, inputs are summed by a neuron
  - When the input exceeds a threshold the neuron sends an electrical spike that travels from the body, down the axon, to the next neuron(s).
- Synapses or connections between neurons have different conduction strengths that modify the input values. In other words, the adding of synpses is weighted.
- Synapses can be inhibitory or excitatory.

# Artificial Neural Networks

- McCulloch & Pitts (1943) are generally recognized as the designers of the first artificial neural network.
- Many of their ideas are still used today, e.g.,
  - Many simple units, “neurons” combine to give increased computational power.
  - Some connections are positively weighted, others negatively.
  - They introduced the idea of a threshold needed for activation of a neuron.



# Modelling a Neuron

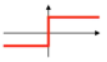
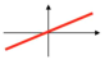
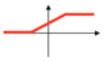
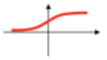
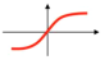
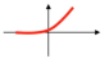



From towardsdatascience.com

# Characterizing an ANN

- Activation Function at a single neuron level – Step, sign, sigmoid, RelU, etc.
- Architecture at network level – How are the neurons organized, how are they connected?
- Learning Algorithm at network level – How connection weights are changed or learned over time? It is standardly backpropagation algorithm, which is a gradient descent algorithm; it is used with various optimization functions.

# Examples of Activation Functions

| Activation function | Equation | Example | 1D Graph |
|---|---|---|---|
| Unit step (Heaviside) | $\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$ | Perceptron variant |  |
| Sign (Signum) | $\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$ | Perceptron variant |  |
| Linear | $\phi(z) = z$ | Adaline, linear regression |  |
| Piece-wise linear | $\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$ | Support vector machine |  |
| Logistic (sigmoid) | $\phi(z) = \frac{1}{1 + e^{-z}}$ | Logistic regression, Multi-layer NN |  |
| Hyperbolic tangent | $\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ | Multi-layer Neural Networks |  |
| Rectifier, ReLU (Rectified Linear Unit) | $\phi(z) = max(0, z)$ | Multi-layer Neural Networks |  |
| Rectifier, softplus | $\phi(z) = \ln(1 + e^z)$ | Multi-layer Neural Networks |  |



# Learning: Setting the Weight

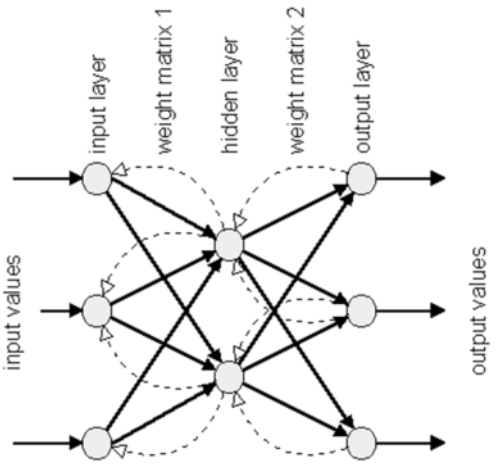
- Supervised: Classification, provide a bunch of pre-labeled or pre-classified examples. That is, each example is a pair <input (in terms of features), label>.
- Unsupervised: No labeled examples. Group instances based on “inherent similarity”. Examples: Clustering.

# Learning: Rules

- Whether in supervised or unsupervised learning, we need rules to update the weights so that with enough examples or iterations, they become (close to) optimal in mapping the input values to the corresponding output values.
- *Backpropagation Learning Rule*: Change the weights so that the root mean squared error (RMSE) or some other error function (e.g., Cross-Entropy Loss) is minimized between expected output and produced output.

# Learning: Backpropagation

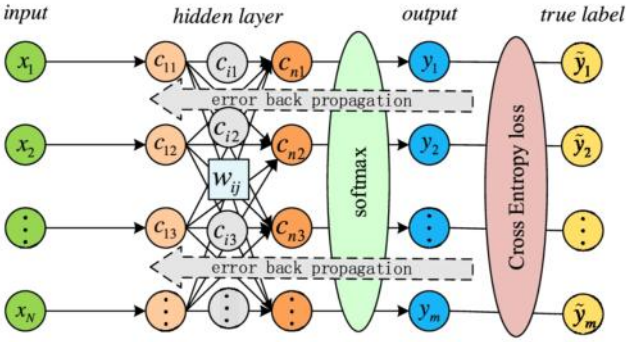
- Backpropagation was introduced in the 1980s and revived ANNs, which was "dying" at that point because the learning rules allowed learning only linear functions, or were "arbitrary".
- The training of a network by backpropagation involves three stages.
- The feedforward of the input training pattern, i.e., the forward calculations; computation of error at output layer nodes; and the adjustment of the weights by backward propagation of the error.
- After training, application of the net involves only the computations of the feedforward phase.



# Learning: Backpropagation

- In other words, the network is shown a lot of (input, output) pairs.
- Initially the weights are random or heuristically obtained.
- When the output produced by the network doesn't match the expected output, feedback is sent back to adjust the weights a little. *The error is called Loss*.
- There are various loss functions such as *Mean Squared Loss* and *CrossEntropy Loss*.
- The weights are changed using a gradient descent algorithm, *based on the loss.*
- With enough training examples, the weights arrive at optimized or close to optimized values.

# Learning: Loss Function




When an input is shown to the network during training, it usually makes errors in producing outputs. So, the output is also a set of numbers, which are likely to be wrong. The ANN computes the difference or error between the outputs obtained and outputs that are are expected. This is loss used for backpropagation.

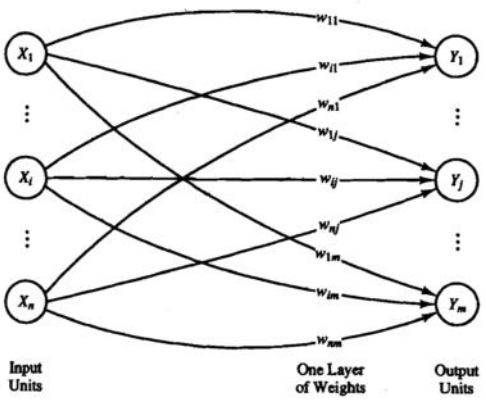
www.semanticscholar.org

# Architectures

- We will briefly look at several popular architectures in this lecture, without much details.
  - Single-layer feed-forward networks (for historical reasons, also used as a subnetwork often)
  - Multi-layer feed-forward networks, also called perceptrons (used a subnetwork often)
  - Convolutional neural networks
  - Recurrent neural networks
  - Attention in neural networks
  - Transformers

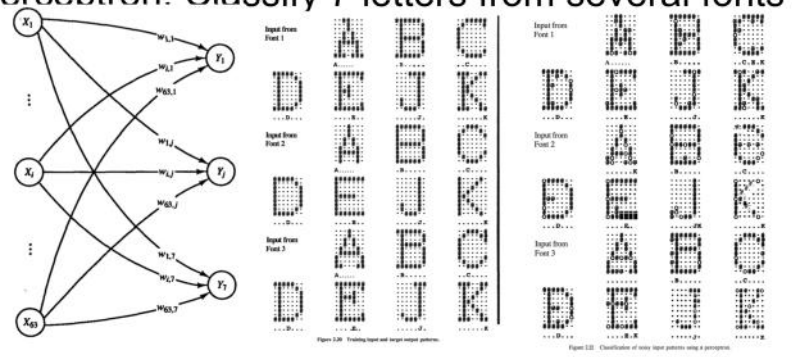
# Dense Neural Networks

# Architecture: Single Layer Feedforward ANN or Perceptron




Fundamentals of Neural Networks: Architectures, Algorithms And Applications by Laurene V. Fausett, 1990

# Perceptron: Classify 7 letters from several fonts



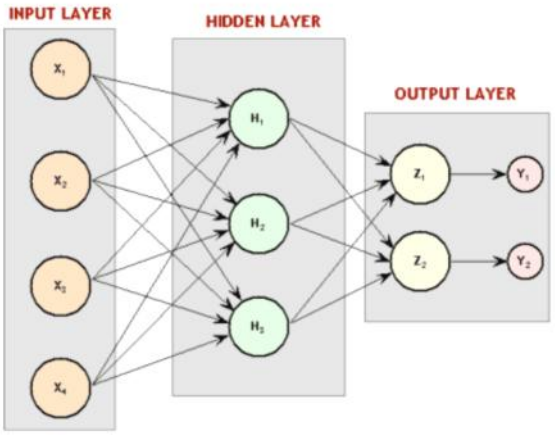
Training input with target patterns

Classification of noisy inputs using perceptron

Fundamentals of Neural Networks: Architectures, Algorithms And Applications by Laurene V. Fausett. 1990

# Architecture: Multilayer Neural Network

- There may be one or more hidden layers




This is a so-called dense neural network where each node in a layer is connected to every node in the previous layer.

For a long time, till the late 2000s, it was thought that one or two densely connected hidden layers are enough for any neural network. It is theoretically TRUE!

It was also rigorously proved that such a neural network is a universal function approximator.

# Convolutional Neural Networks

## Architecture: Convolutional Neural Networks

- In biological systems, neurons of similar functionality are usually organized in separate areas (or layers).
- Often, there is a hierarchy of interconnected layers with the lowest layer receiving sensory input and neurons in higher layers computing more complex functions.
- However, in the human visual and other systems, a node at a higher level usually gathers input from only a few nodes in the previous layer. This is unlike a dense neural network we saw earlier.

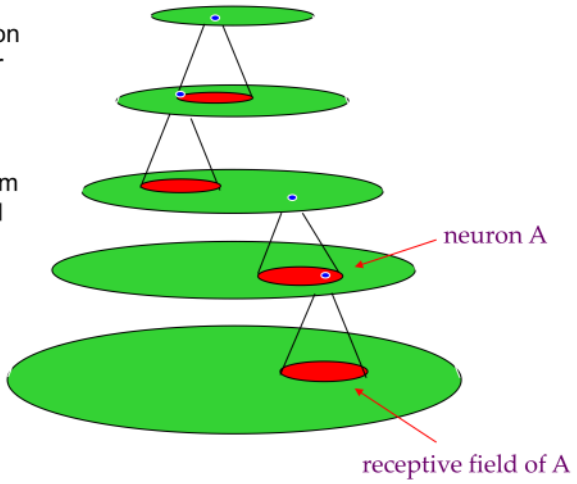


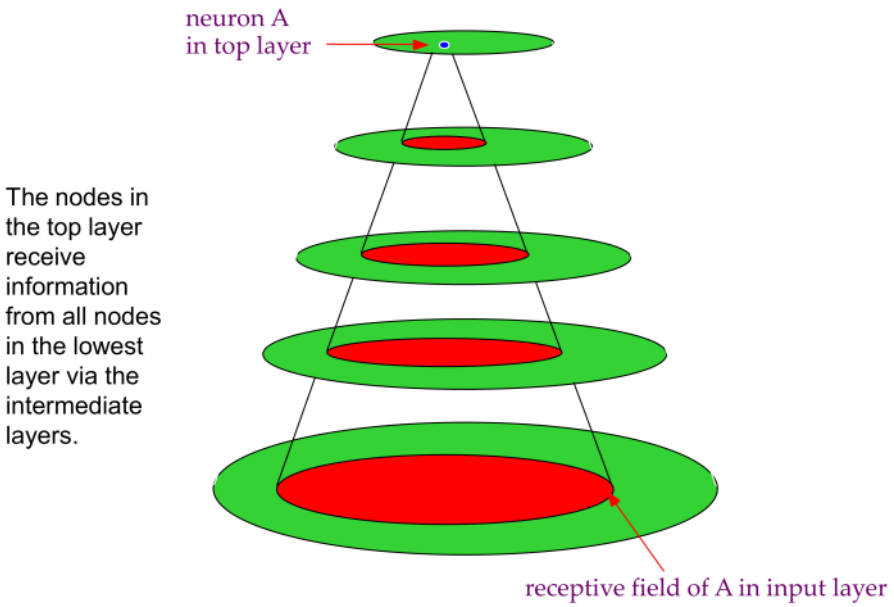


# Receptive Fields in Vision

A node in an upper layer collects and processes information from a small number of nodes in a lower layer.

The set of nodes from which an upper level node receives information from a lower layer, is called its receptive field.








# Modeling the Visual System

- The visual system is modeled as a hierarchy.
- Simple features in lower layers
- Complex features in higher layers, gathering more overall/spread-out information

# Convolutions in Computer Vision

35

- Lower processing units in the visual system extract low level features, and higher processing units extract high level features.
- Many convolutionsor small transformation matrices were designed in computer vision to extract features from images before passing them on for further computation.

**Simple Box Blur**

It has an averaging effect, resulting in a slight blur.

| | | |
|---|---|---|
| 1/9 | 1/9 | 1/9 |
| 1/9 | 1/9 | 1/9 |
| 1/9 | 1/9 | 1/9 |

# Convolutions in Computer Vision

36

Convolutions that detect lines

| | | |
|---|---|---|
| -1 | -1 | -1 |
| 2 | 2 | 2 |
| -1 | -1 | -1 |

Horizontal lines

| | | |
|---|---|---|
| -1 | 2 | -1 |
| -1 | 2 | -1 |
| -1 | 2 | -1 |

Vertical lines

| | | |
|---|---|---|
| -1 | -1 | 2 |
| -1 | 2 | -1 |
| 2 | -1 | -1 |

45 degree lines

| | | |
|---|---|---|
| 2 | -1 | -1 |
| -1 | 2 | -1 |
| -1 | -1 | 2 |

135 degree lines

Detecting horizontal lines

# Convolutions in Computer Vision



**Edge Detection**: Find edges no matter what their orientation are, not just horizontal, vertical or diagonal.

One way is to add up the four convolutions in the previous slide.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

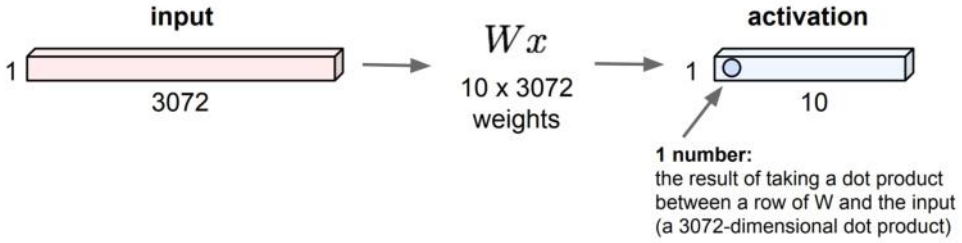
# Convolutions in Computer Vision



- Many such convolutions were devised by researchers in computer vision based on years of work.
- The convolution operation simply multiplies the pixel values by the numbers in the convolution and adds the products to obtain a single number.
- The convolution is moved over the image by dragging it vertically and horizontally using some strides.
- The same thing is done in Convolutional Neural Networks (CNNs)—we use a number of convolutions, but the values are not determined by researchers, instead they are learned based on inputs and outputs of a training set.
- Lower convolutions extract low-level features, and higher convolutions extract high-level features.

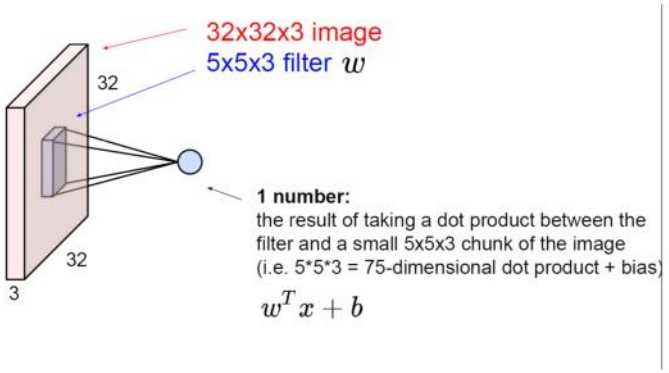
# A dense feedforward network layer

32x32x3 image -> stretch to 3072 x 1



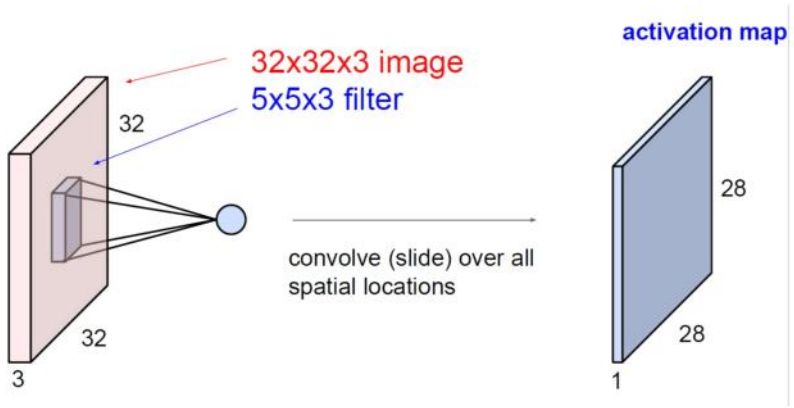

We have a 32 x 32 x 3 image. We have vectorized it into a 3072 long vector. It is fully connected to an output layer which has ten nodes. The weight matrix is 10 x 3072 in size. In other words, we have 30,720 weights to learn.

# Introducing Receptive Fields: Convolutions



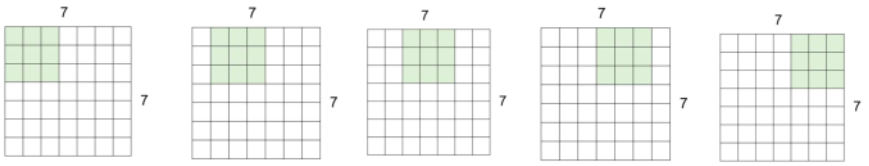

A convolution is a small filter that is applied at points on the image and then slid over the image. If our image is 32 x 32 x 3 (32x32 image with RGB depth 3), we can have a convolution of 5 x 5 x 3 where 3 is the depth. The application of the convolution performs a point-by-point “dot” product called the Hadamard product to produce one single number.

# Slide or convolve the filter/convolution



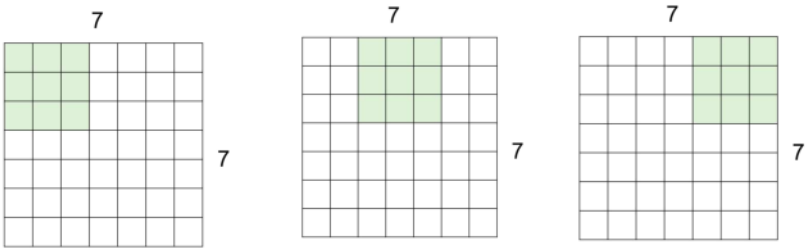

If we slide the convolution or filter over the entire volume of the image, we get a 28x28 activation map, which is simply writes out the activations we get at each position of the image. We will discuss how we obtain the size of the activation map a little later.

# Dimensions of Input/Output Volumes




**Stride of 1**: Suppose our original image is 7x7 (N=7). Assume also that the filter is 3x3 (F=3). We do not show the depth for both the original image and the filter; they are both the same. We move the filter by one location at a time, covering the entire input. The amount of movement from one application of the filter to the next is called Stride. Assume our stride is 1, both horizontally and vertically. The output has size 5x5 since we can place the filter 5 ways each, both horizontally and vertically.

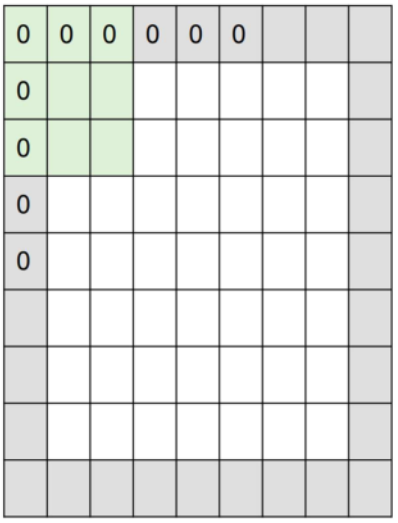
# Dimensions of Input/Output Volumes




**Stride of 2**: With an original image of size 7x7 and filter of size 3x3, a stride of 2 both horizontally and vertically leads to an output of size 3x3.

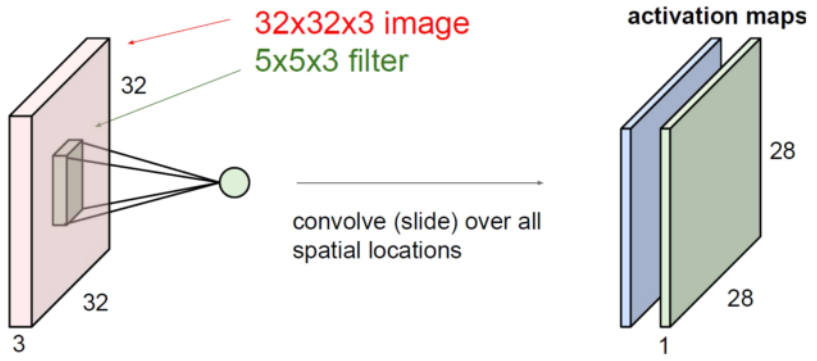
We should be careful to not to use a big stride w.r.t. original image size because this may reduce the size of the output too quickly, especially if we have a several convolutional layers. We may lose details too quickly.

# Padding



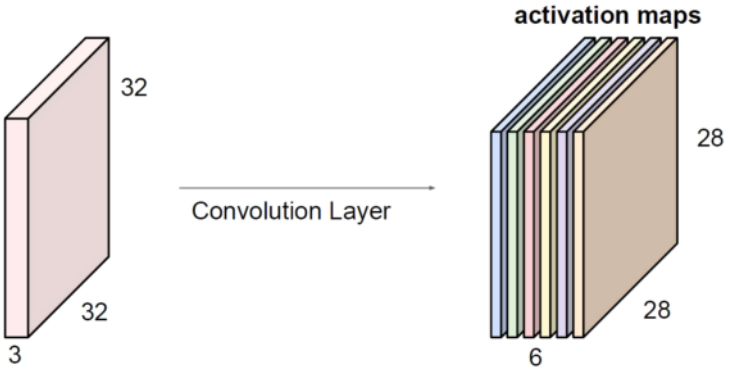

We can insert some dummy elements on the sides, top and bottom to make the filter fit. A simple but reasonable way to pad is to imagine there is a bunch of 0s in rows above and below, and columns on the right and left edges. The number of padded rows/columns, can be easily computed.

# Using two convolutions



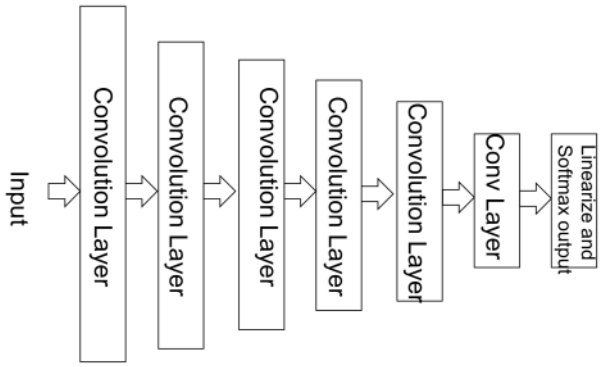

We can have a second convolution or filter and slide it also all over the image volume to obtain a second activation map. The filters may be of the same size, but initialized randomly in an independent manner. The two filters may be able to extract two different pieces of local information from the image, because the filter values are randomly initialized and then learned.

# Have several convolutions




We can use a number of different convolutions/filters of the same size (or different sizes also) and apply them to the image one by one or in parallel, and obtain a number of activation maps. In this example, we use six different filters to obtain 6 activation maps, each 28x28 in size. We can think of these 6 activation maps together as another representation of the input image. The number of filters used is usually a small number like 2, 3, 5 or 10, but in some cases, can be large.

# Basic Convolutional Neural Network




A basic convolutional neural network simply consists of an input layer, and a number of convolutional layers, and a fully connected output layer. So, it can look like the one we have above. In practice, it has other layers such as pooling and dropout, interspersed with the convolution layers.
The last convolution layer's output has to be linearized before passing to a softmax layer for output, if it is a classifier. It doesn't need the softmax for a regression network.

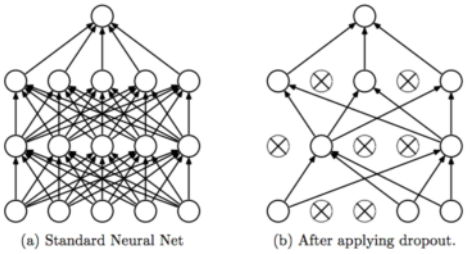
# Other Layers

It was found that too many convolutional layers alone makes it difficult to generalize, to get good results.

It has been found that it helps to interleave the convolutional neural network with

- dropout layers,
- and pooling layers.

# Dropout Layer



(a) Standard Neural Net (b) After applying dropout.

Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting", JMLR 2014

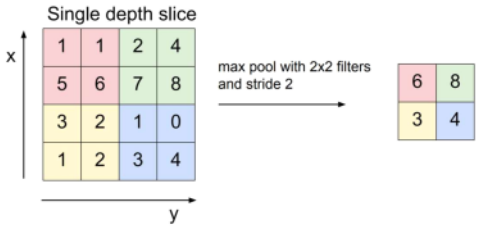
Dropout forces a neural network to learn more robust features that are useful in conjunction with many different random subsets of the other neurons.

Dropout increases the number of iterations required to converge. However, training time for each epoch is less.

**Training Phase**:
Training Phase: Ignore (zero out) a random fraction $p$ of nodes (and corresponding activations) at every iteration of training, for every example, in every layer.

**Testing Phase**:
Use all activations, but reduce them by a factor $p$ (to account for the missing activations during training).

# Downsampling: Pooling Layer

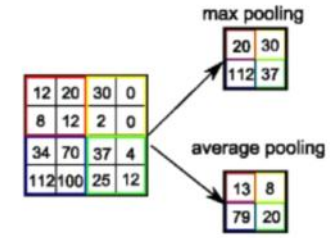



Pooling looks at a certain rectangular region in the image and performs a simple computation like averaging, or picking the largest value.

Max pooling is quite commonly used. Max pooling allows for certain locational variation on features. E.g., if a line is not exactly straight but is a slightly broken, it may be picked up by the use of max pooling.

Max pooling is used when we want some prominent features to stand out. Average pooling may be used when we know that there are noises, and we want to smooth them out.

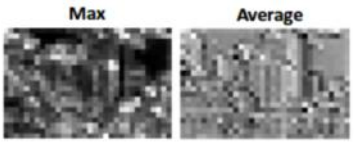
# Downsampling: Pooling Layer



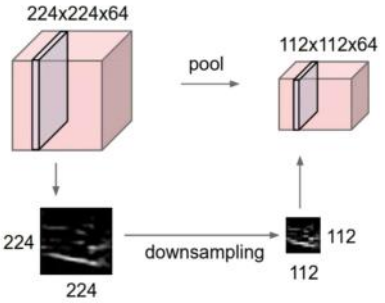

Max pooling extracts the most important features like edges whereas, average pooling extracts features smoothly.

For image data, you can see the difference.

Although both are used depending on need, max pooling is more common.

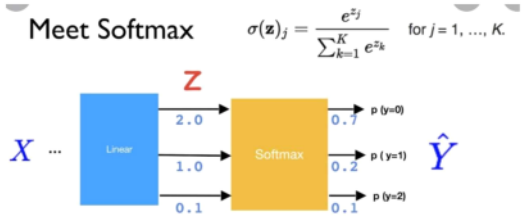



# Downsampling: Pooling Layer




Pooling makes the representations smaller and more manageable.
Pooling captures certain simple deviations from the normal, within a small region of the image. For example, if a bright pixel of a certain color was expected in a certain spot, but it appears displaced by one spot.
Pooling operates over each activation map independently.

# Usual Last Layer for Classification: Softmax




The softmax function is usually used as the activation function in the last layer or classification layer.

The softmax function takes the activation of the penultimate layer, obtained by extracting the highest level features, and then "squashes" them to make them "probabilities", i.e., the total is 1, and every number is between 0 and 1.

Here is a link to a softmax calculator: https://www.redcrab-software.com/en/Calculator/Softmax

# Basic Convolutional Neural Network

The convolution layer performs the component by component product (Hadamard product) of the inputs with the weights on the convolution, and then passes it to an activation function.

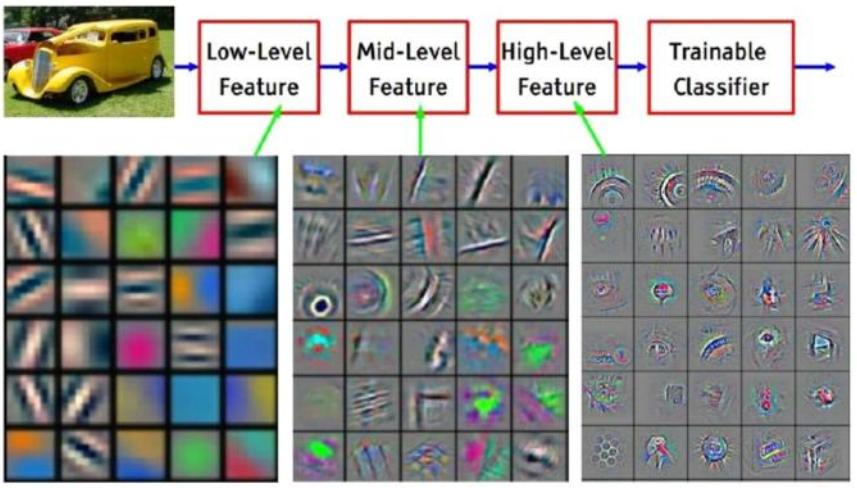
For a long time, the sigmoid activation was used.

But, starting 2014 or so, the ReLU activation has been more common in hidden/inside layers.

There is usually one or more dense layers on top of the convolutional/pooling/dropout layers.

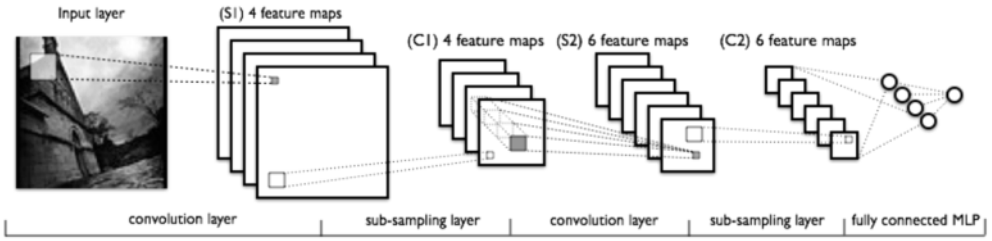
The softmax action function is used in the output layer or classification layer, for a classification network.

# Layers capture features: Simple to Complex




Zeiler and Fergus, 2013, Feature visualization of Convolutional Net trained on ImageNet

# LeNet: First CNN Architecture




The architecture has convolutional layer, followed by a pooling/sub-sampling layer, another convolutional layer, followed by a pooling layer, and a fully connected layer.

From http://deeplearning.net/tutorial/lenet.html

LeCun, Bottou, Bengio, and Haffner, 1998, Gradient-based learning applied to document recognition, Proc. IEEE 86(11): 2278–2324, 1998.

# ImageNet & Competitions

ImageNet is an image database organized according to the WordNet hierarchy (currently only the nouns), in which each node of the hierarchy is depicted by hundreds and thousands of images. Currently it has an average of over five hundred images per node.

WordNet is a lexical database for the English language. It groups English words into sets of synonyms called synsets, provides short definitions and usage examples, and records a number of relations among these synonym sets or their members. The latest version has 115K unique words, organized into 176K synsets. There are 81K noun synsets. Each node in the graph is a synset.


http://www.cs.princeton.edu/courses/archive/spring07/cos226/assignments/wordnet.html

# ImageNet & Competitions

ImageNet has

~14 million labeled images, 20k classes
Images gathered from Internet
Human labels via Amazon Mechanical Turk

ImageNet Large-Scale Visual Recognition Challenge (ILSVRC): 1.2 million training images, 1000 classes


http://www.cs.princeton.edu/courses/archive/spring07/cos226/assignments/wordnet.html

# AlexNet: ILSVRC Winner 2012

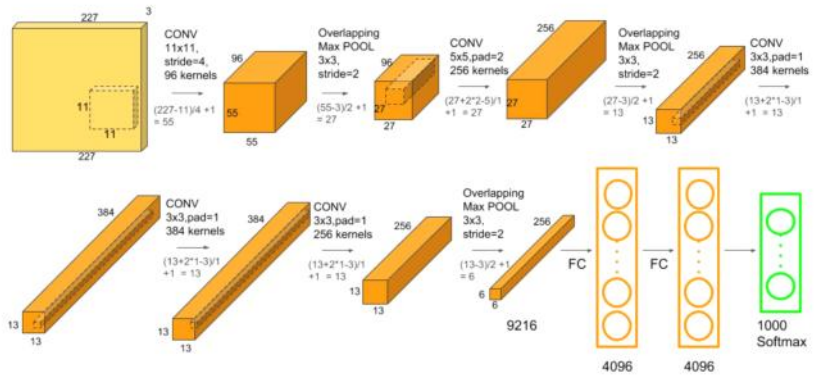
AlexNet is the name of a convolutional neural network, originally written with CUDA to run with GPU support, which competed in the ImageNet Large Scale Visual Recognition Challenge in 2012. The network achieved a top-5 error of 15.3%, more than 10.8 percentage points ahead of the runner up.

Used Max pooling, ReLU nonlinearity; 7 hidden layers, 650K units, Dropout regularization. *Had 60 million parameters*.

Krizhevsky, A., I. Sutskever, and G. Hinton, ImageNet Classification with Deep Convolutional Neural Networks, Neural Information Processing Systems, 2012; cited 152,963 times as of 3/5/2026

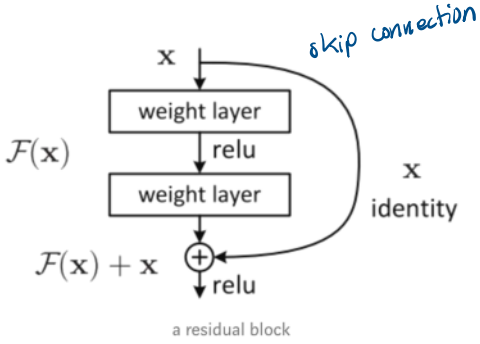
# AlexNet: ILSVRC Winner 2012

changed how NN are built




https://neurohive.io/en/popular-networks/alexnet-imagenet-classification-with-deep-convolutional-neural-networks/

# ResNet: ILSVRC Winner 2015



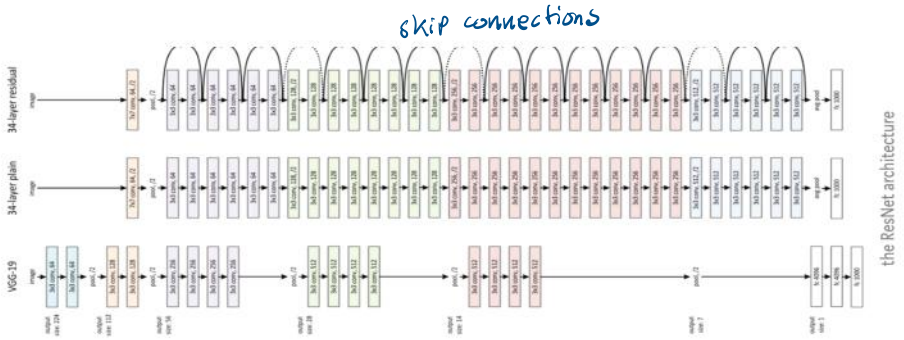

Deep neural networks suffer from the problem of vanishing gradients.

Since learning of weights depends on the computation of gradients of the loss function w.r.t. to the individual weights, sometimes, the gradient becomes too small and learning does not take place.

Residual network is an attempt to reduce the effect of vanishing gradient.

Connecting across layers feeds activation from a lower unit to a higher unit two or more layers away.

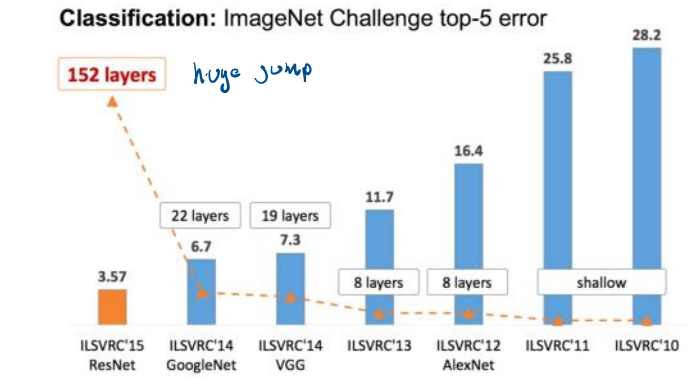
# ResNet: ILSVRC Winner 2015




The bottom one is a CNN called VGG-19, the middle one has 34 layers (plain).

The top one has 34 layers, but also residual connections.

# ILSVRC Winner 2010-15




http://slazebni.cs.illinois.edu/spring17/lec01_cnn_architectures.pdf

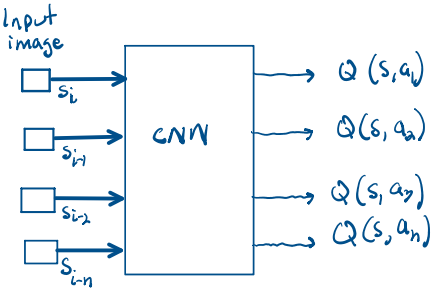
# ILSVRC Winner 2016

The 2016 winner uses an ensemble of 6 CNNs.

An ensemble runs several programs in parallel and combines their results by voting, weighted voting or some other means.

The ILVRC Competition which started in 2010 was discontinued in 2017, but it had some of the greatest contributions to advancing machine learning

Current high-performing models on ImageNet can be found here: https://hiringnet.com/image-classification-state-of-the-art-models-in-2025



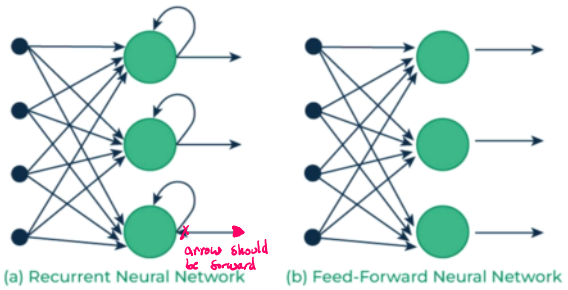

Sequence prediction problem?

# Recurrent Neural Networks

# Recurrent Neural Networks

Feedforward ANNs like MLPs or CNNs take one input and produce one output.

Recurrent neural networks, also known as RNNs, are a class of neural networks that allow previous outputs to be used as inputs, i.e., have a feedback loop.




geeksforgeeks.org

# Types of RNN

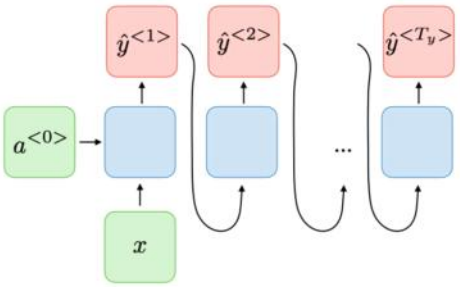
One-to-many (Story generation, given a topic; poem generation, given a title)

Many-to-one (Sentiment classification; story classification)

Many-to-many (Named entity recognition; POS tagging)

Many-to-many (Machine translation)

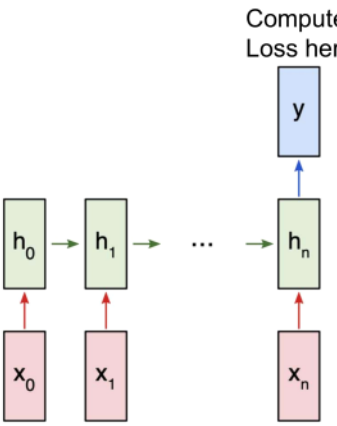
# RNN: One-to-many




Story generation, given a topic; poem generation, given a title

https://stanford.edu/~shervine/teaching/cs-230

# Many-to-one RNN




Here, an RNN takes a number of input words in a sentence, processes it one by one, form an increasingly complex representation of the input and classifies the final representation.

The same idea can be used for reinforcement learning as well:

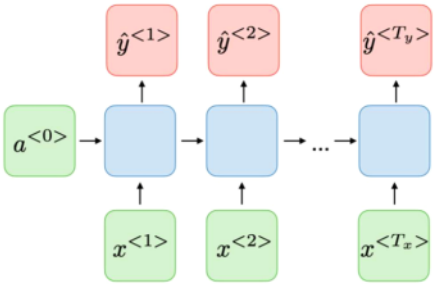
*Given a sequence of S, A, R, S, A, R, …, predict the next S or predict the next R or predict the next A. How to do this will require work.*

The loss function will have to be appropriately defined for various deep RL algorithms.

victorzhou.com

# RNN: Many-to-many

learn in one env.
act in another env

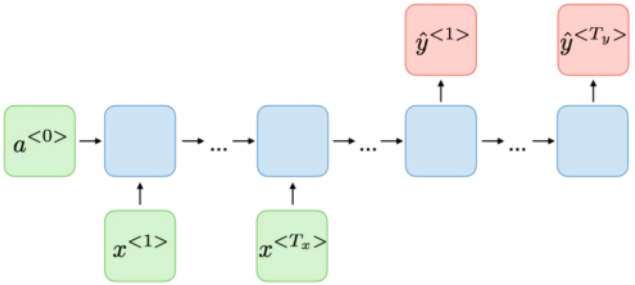



Pos tagging; Named entity recognition

Off-policy learning or transfer learning in RL: Given a sequence SARSAR… in one domain, predict SARSAR…in another related domain.

https://stanford.edu/~shervine/teaching/cs-230

# RNN: Many-to-many

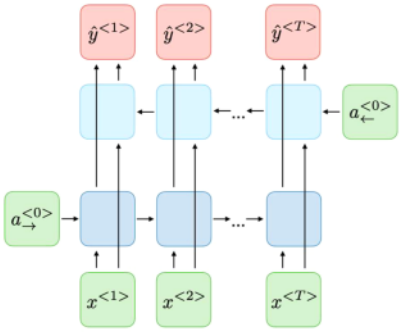


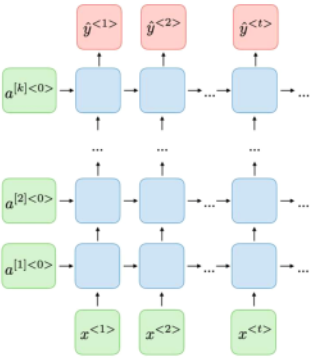

Machine translation; Question answering

https://stanford.edu/~shervine/teaching/cs-230

Transfer learning: Given SARSAR… in one domain, predict SARSAR…in another domain. Depends on how one wants to do it.

# RNN Variants




Bidirectional RNN

Deep RNN with many layers

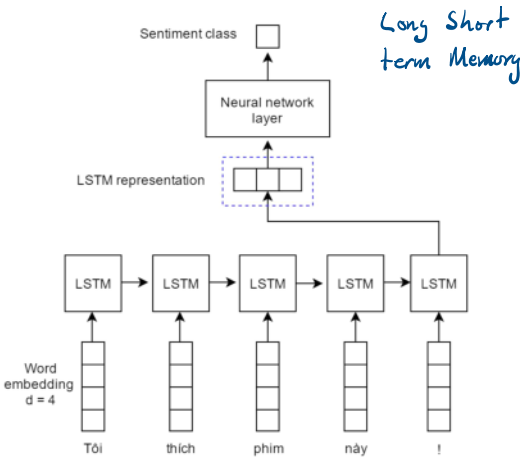
https://stanford.edu/~shervine/teaching/cs-230

# Performing Classification with RNNs

Use an RNN to obtain a representation for the sentence or the piece of text.

Use another neural network like a Multi-layer Perceptron or CNN to perform classification.

LSTM is a kind of RNN, but with longer short-term memory.




https://www.researchgate.net/figure/Illustration-of-our-LSTM-model-for-sentiment-classification-Each-word-is-transfered-to-a_fig2_321259272

# Problem with RNNs

Training of RNNs, like any Neural Network, happens via backpropagation, which requires computation of gradients of loss for each edge weight at all levels in the neural network.

In an RNN, backpropagation is performed on the unrolled network, the gradient of the loss (which is computed at the end) may vanish.

Vanishing gradient causes the RNN not to learn.

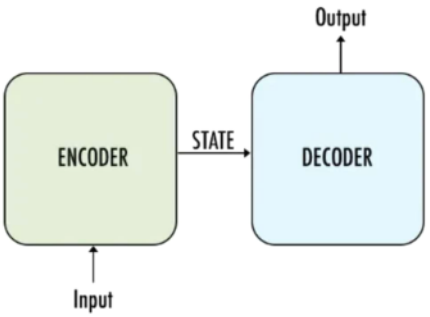
As a result, an RNN is not able to pick up well on long-distance dependencies in a sequence of input.

In the case of RL, the paths to be learned can be long, and/or the rewards may be rare after long sequences. Whether we use regular RNNs or variants like LSTM, the results may not be great.

This leads to the idea of attention in a neural network.

# Encoder-Decoder Architecture

State or representation
compact representation

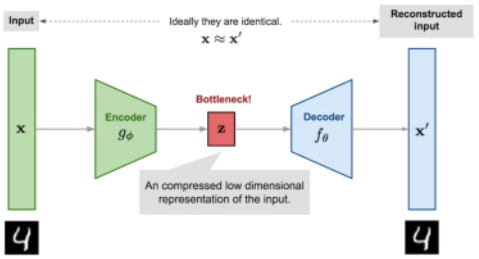



From medium.com

An encoder-decoder architecture consists of two neural networks put together and trained together. The encoder takes an input and learns to produce an internal output, here called state or code. This internal output is fed to the decoder to produce an output. The encoder and decoder architectures are trained together, so that the entire architecture learns to produce a desired output given an input.

# Encoder-Decoder Architecture

low res image to high res




From eugeneyan.com

In an encoder-decoder architecture, the ``code" or the encoded latent representation is often called the *bottleneck* since it is much smaller than the input.

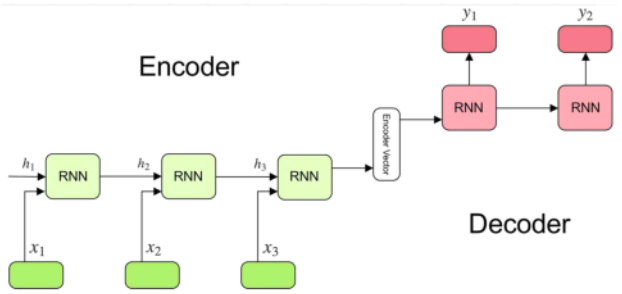
For example, the bottleneck can be 10 nodes whereas the input is 28 x 28 = 784 size here. The code is usually quite small. The code must capture the most essential features so the input can be reconstructed as something similar, better or worse. The input also can be the entire sequence of words in a piece of text or "words" in an SARSAR… sequence. The bottleneck is then the compact representation.

In the architecture given above, the architecture learns to take an input **x,** send it through an encoder CNN to produce the code **z,** which is then passed through a reverse CNN to produce a variation of it called **x'**. For example, **x'** may be a super-resolution or cleaner version of **x**.

# Machine Translation Has Driven Most Recent Innovations in Artificial Neural Networks

- Initial Machine Translation work was using rules and what are called n-grams, and statistical computation.
- Initial work using Neural Networks used Encoder-Decoder Architectures with RNNs (LSTMs), like Sutskever et al. (2014).
- Next, came the attention mechanism, as in Bahdanau et al. (2014)
- The “final” innovation was the Transformer architecture of Vaswani et al. (2017).

# Encoder-Decoder Architecture

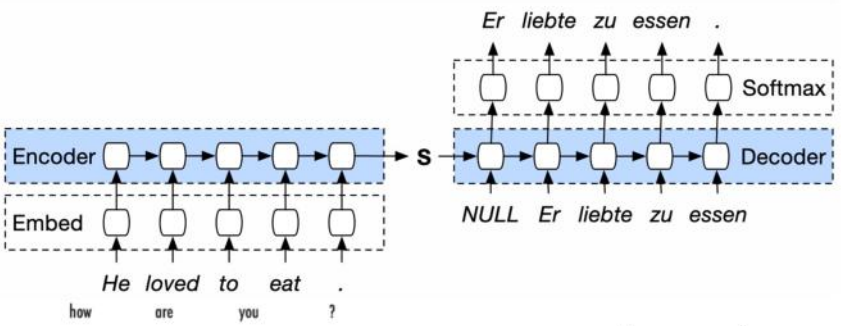



From medium.com

The idea of encoder-decoder architectures comes from machine translation.

There is an RNN encoder to process the input to produce an encoded version of the input, which is then processed by the decoder to produce the output.

# Encoder-Decoder for Translation




From smerity.com

The original application of encoder-decoder architecture is for machine translation. (In RL, it can be a SARSAR…sequence to another SARSAR…sequence, as in transfer learning in RL).

**Issue**: The input, which can be long and can have short or long-term dependencies, needs to be encoded to a small ``code". This is often not enough to produce good responses.

# Alignment in Machine Translatoin

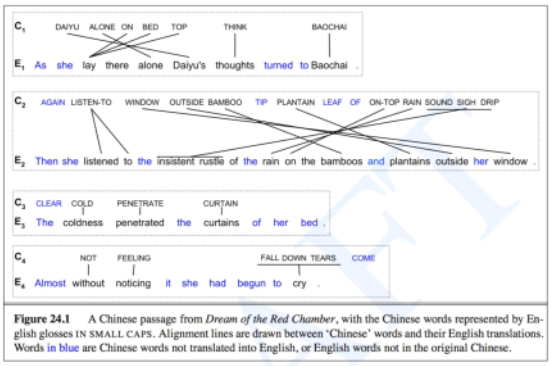



**Figure 24.1** A Chinese passage from *Dream of the Red Chamber*, with the Chinese words represented by English glosses IN SMALL CAPS. Alignment lines are drawn between 'Chinese' words and their English translations. Words in blue are Chinese words not translated into English, or English words not in the original Chinese.

From Jurafsky and Martin's book on Natural Language Processing, 3rd edition.

It's an example the translation from Chinese to English, although the Chinese is given in its English gloss or word by word translation.

In translating a sentence from one language to another, there is a lot of movement of words and phrases. Some languages are SVO, whereas many other languages are SOV or even very loosely word-ordered or not word-ordered at all. How to ensure words move to the right place in the output is called *alignment*.

If we talk about two similar domains in RL, there may be reversal of some subsequences from one to the other.

# Attention in Neural Networks

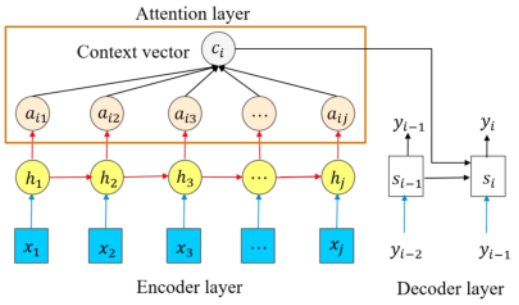
## Attention Mechanism (input-output or cross-attention)

Attention mechanism in an RNN saves the intermediate outputs of the RNN, processes them via a simple MLP, and inputs the output of the MLP into each step of the decoder that produces the output.

If the RNN produces only one output at the very end, the MLP's output is fed to this output step.

This is input-output or cross-attention.

Long-distance dependencies are learned better with attention.



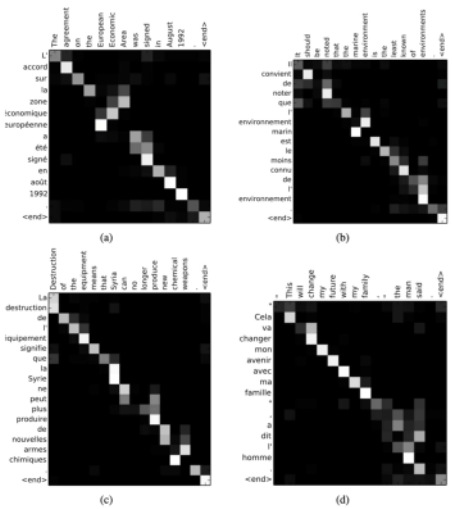

Attention mechanism of Bahdanau

Illustration from Yang, Li and Liu, *Electronics* 2021,*10*,1657

hidden layers are saved. This attention layer is then feed into the decoder along with the vector output.

cross-attention - looking back at the input for context, or matter the most. Better translation.

# Cross-Attention in Translation




***Cross-Attention***: Relate one or more output words to one or more input words.

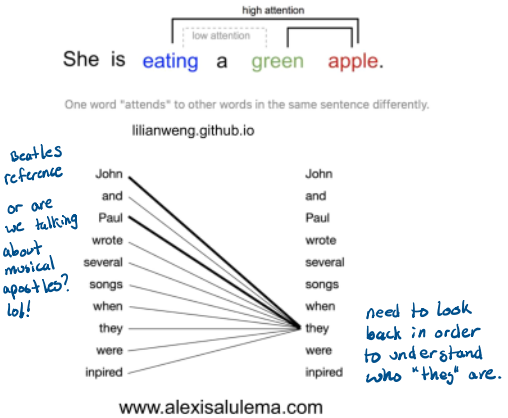
RNN with traditional attention mechanism does this reasonably well.

Bahdanau et al., 2016. Neural Machine Translation by Jointly Learning to Align and Translate.

# Going Beyond Cross-attention: Self-Attention and Transformers

Transformer brings in Self-attention

## Self-Attention




Beatles reference or are we talking about musical apostles? lol!

need to look back in order to understand who "they" are.

***Self-Attention***: In addition to input-output attention, *we* also want to relate one or more words in one sequence to words in the same sequence (input or output).

The diagram shows that when "understanding" the word *they*, there is high attention paid back to the words *John* and *Paul*.

In RL, there may be elements in an SARSAR…sequence that are "related"/co-related to other elements in the sequence.

# Transformer at a High Level

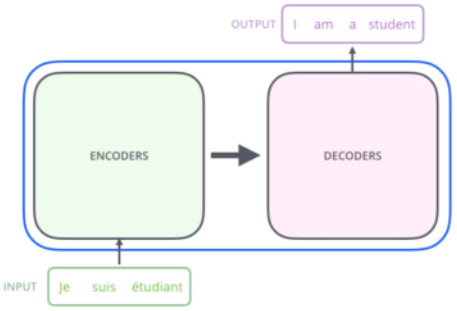
Just a NN


http://jalammar.github.io/illustrated-transformer/

At a very high level, the Transformer takes an input sequence and converts to another sequence, just like an RNN encoder-decoder with or without attention.

In this case, it's performing translation from French to English.

# Transformer is an Encoder-Decoder Model




http://jalammar.github.io/illustrated-transformer/

The encoder forms a representation of the input sequence.
The decoder generates an output sequence from the encoded input.

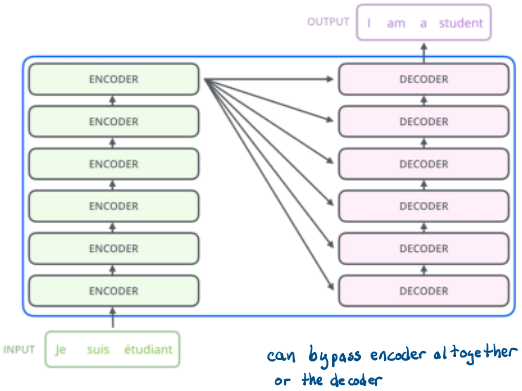
# Attention and Transformers

## Transformer:

There is a stack of identical decoders, and a stack of identical encoders, 6 of each in the original architecture.

The encoder output at the top is fed to every level of the decoder stack.

Thus, a higher level decoder produces an (intermediate) output based lower level output and a full “understanding” of the input.

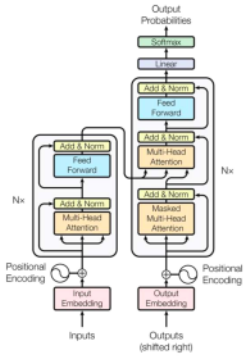



each stage gets the original input for context

can bypass encoder altogether or the decoder

# Transformer at a High Level

## Transformer in its Entirety



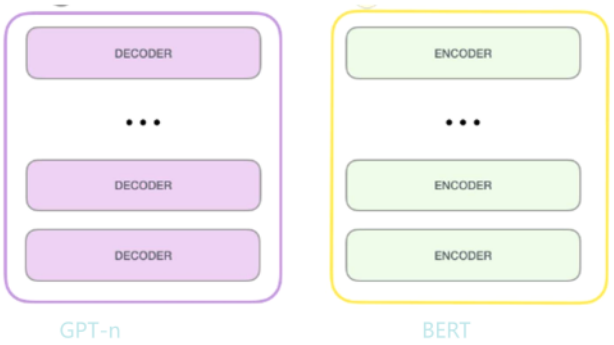

Vaswani et al. 2017

- The Transformer architecture is a complex encoder-decoder architecture.
- It is trained on (input, output) pairs where each is a sequence.
- Unlike an RNN, it is not sequential.
- Like a CNN, it takes the entire input and output at the same time as it is trained. Usually 512 tokens.
- The input is entered in full or divided into blocks each of which can be entered in full.
- However, Transformer is much faster.
- Unlike a CNN, it uses “position encoding” to know which word is in which position of the input. CNN processes all inputs similarly without noting their positions or locations within the input.
- Transformers form the basis of all state of the art neural network methods whether for NLP or vision or most any other domain.

- (Late 2025) For example, OpenAI’s GPT-3.5-turbo supports up to 16,384 tokens, while GPT-4 offers variants with 8,192 or 32,768 tokens. Other models, like Anthropic’s Claude 2, extend this to 100,000 tokens. These limits include both input and output, so longer prompts reduce the space available for responses. (https://milvus.io/ai-quick-reference/what-is-the-maximum-input-length-an-llm-can-handle)

# Attention and Transformers

## Encoder-Only & Decoder-Only Transformers




# Encoder-Only Transformer Examples

These models are specialized for understanding, classification, and embedding tasks.

BERT (Bidirectional Encoder Representations from Transformers): The foundational model for understanding language context.

# Decoder-Only Transformer Examples

These models are the standard for generative AI, creating text token-by-token.

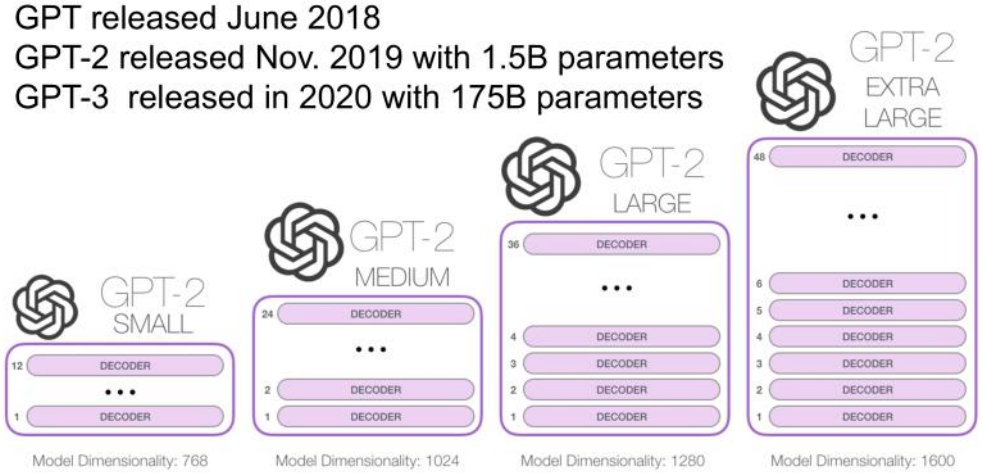
Examples:
- GPT Series (GPT-3, GPT-4, GPT-5): OpenAI's generative pre-trained transformers.
- LLaMA (Large Language Model Meta AI): Meta's open-source decoder-only models.
- Gemini: Google's multimodal decoder-only models.
- Claude: Anthropic's series of generative models.

Key Differences

•Encoder-Only: Uses bidirectional attention (looks at left and right context simultaneously).
•Decoder-Only: Uses masked/unidirectional attention (only looks at previous tokens to predict the next).

all based on transformers

GPT released June 2018
GPT-2 released Nov. 2019 with 1.5B parameters
GPT-3 released in 2020 with 175B parameters




Model Dimensionality: 768 | Model Dimensionality: 1024 | Model Dimensionality: 1280 | Model Dimensionality: 1600

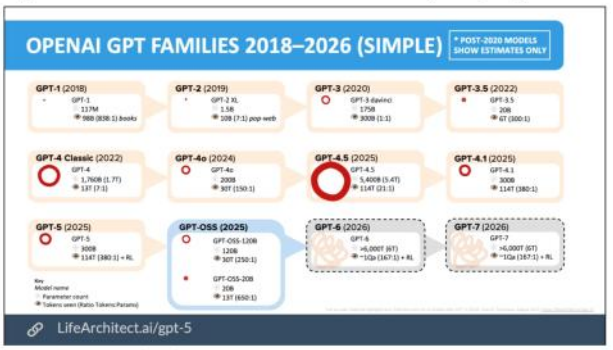
117M parameters | 345M | 762M | 1542M

## Comparing OpenAI GPT products

Accessed 11/12/2025

Later versions of GPT starting GPT-3 have used Reinforcement Learning with Human Feedback (RLHF) to produce better results.

OpenAI GPT families 2018–2026 (simple)




https://lifearchitect.ai/gpt-5/

## GPT-5 Summary

Accessed 11/12/2025

Summary

| | |
|---|---|
| Organization | OpenAI |
| Model name | GPT-5 |
| Internal/project name | Gobi, Arrakis |
| Model type | Multimodal |
| Parameter count | ~300B. See *What's in GPT-5?* |
| Dataset size (tokens) | ~114T. See *What's in GPT-5?* |
| Training data end date | Sep/2024 |
| Training start date | Jan/2025 |
| Training end/convergence date | Apr/2025 |
| Training time (total) | |
| Release date (public) | 7/Aug/2025 |
| Paper | https://openai.com/index/gpt-5-system-card/ |

| Model | Training end | Chip type | Chip count | Wall clock (days) | Total time (years) | Retail ($/1E) | MMLU |
|---|---|---|---|---|---|---|---|
| GPT-3 175B | Apr/2020 | V100 | 10,000 | 15 days | 405y | $5M | 43.9 |
| Llama 1 65B | Jan/2023 | A100 | 2,048 | 21 days | 118y | $4M | 63.4 |
| Llama 2 70B | Jun/2023 | A100 | 2,048 | 35 days | 196y | $7M | 68.0 |
| Titan 200B | Apr/2023 | A100 | 13,760 | 48 days | 1,815y | $45M | 70.4 |
| GPT-4 1.7T | Aug/2022 | A100 | 25,000 | 90 days | 6,507y | $224M | 86.4 |
| Gemini | Nov/2023 | TPUv4 | 57,600 | 100 days | 15,605y | $440M | 90.0 |
| Llama 3 405B | Apr/2024 | H100 | 24,576 | 50 days | 3,366y | $125M | 85+ |
| GPT-5 | Apr/2024 | H100 | 30,000 | 120 days | 9,438y | $810M | |
| Grok 2 | Jun/2024 | H100 | 20,000 | 60 days | 4,107y | $240M | |
| Olympus | Aug/2024 | H100 | 989 | | | | |
| Gemini 2 | Nov/2024 | TPUv6 | 1,847 | | | | |
| Grok 3 | Dec/2024 | H100 | 100,000 | 60 days | 32,855y | $1.2B | |

Table. Model training compute (see working, with sources).

See working, with sources.

https://lifearchitect.ai/gpt-5/

used in RL also

# Decision Transformer (2022)

The transformer can be used to predict actions from a sequence of SAR values.

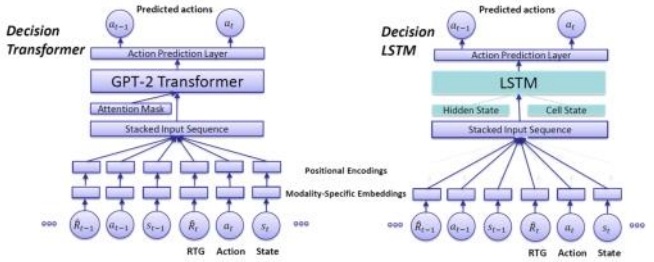



Figure 1: Comparison of the architectures of Decision Transformer (on the left) and the proposed Decision LSTM (on the right). The bottom items show the sequences of observed return-to-go (RTG) values, actions and states, which are used to predict the next action in an auto-regressive way. The key differences between the architectures are the use of an LSTM network as a replacement for the GPT-2 Transformer and the removal of positional encodings for DLSTM.

https://arxiv.org/pdf/2211.14655v1.pdf